

UNIVERSIDAD SANTO TOMÁS

Sistema colaborativo de robots móviles para asistencia de orientación en entornos interiores con múltiples pisos

Realizado por:

Santiago Hernández Ávila
Jonny Alejandro Mejía León



Informe de Avance — Semana 17

Fase 5: Integración del sistema y realización de pruebas — Liberación del vehículo físico y contrato de interfaces ROS 2

Dirigido por:

Ing. Armando Mateus Rojas, Msc.
Ing. Nestor Ivan Ospina, Msc.
Ing. Oscar Mauricio Gélvez Lizarazo, Msc.

GED (Grupo de Estudio y Desarrollo en Robótica)
Facultad de Ingeniería Electrónica
División de Ingenierías
Bogotá, agosto de 2026

1. Introducción

El presente informe documenta el avance correspondiente a la **Semana 17** del cronograma de actividades. Esta semana abre el segundo periodo académico del proyecto y con él la **Fase 5 (Integración del sistema y realización de pruebas)**, tras el receso de mitad de año.

La semana tuvo dos frentes de trabajo. En el frente físico se completó la liberación del vehículo AWS DeepRacer respecto de su *firmware* original y se verificó su locomoción bajo el nuevo entorno de cómputo. En el frente de simulación se consolidó y publicó el stack de navegación autónoma desarrollado durante el primer periodo, y se definió el **contrato de interfaces ROS 2** que habilita el trabajo con dos agentes simultáneos.

Adicionalmente se realizó una replanificación completa del cronograma para el intervalo S17–S32, documentada de forma independiente en el *Cronograma actualizado* [2], y se detectó y corrigió un defecto de trazabilidad en el entorno de compilación que invalida parte de la evidencia cartográfica producida en la semana anterior. Este último punto se documenta de forma explícita en la Sección 7, por su impacto sobre los resultados reportados.

2. Objetivos de la Semana

- Liberar el vehículo AWS DeepRacer del *firmware* original, instalar el entorno de cómputo en la Raspberry Pi 4 y en la tarjeta original, y verificar la locomoción del vehículo.
- Consolidar y publicar en el repositorio del proyecto el stack Nav2 + AMCL migrado a ROS 2 Humble y adecuado a cinemática Ackermann.
- Rectificar el cronograma de actividades para el intervalo S17–S32 sobre el estado real del desarrollo.
- Definir el **contrato de interfaces ROS 2** que rige la integración multi-robot: espacios de nombres, marcos de referencia, acciones y tipos de mensaje.
- Capturar la línea base de comportamiento de un solo robot e instrumentar su verificación de forma automática.
- Iniciar la aplicación del contrato sobre el código de descripción y lanzamiento del vehículo.

3. Liberación y puesta en marcha del vehículo físico

3.1. Instalación del entorno de cómputo

Se instaló la distribución `deepracer-custom-car` sobre dos plataformas de cómputo: una **Raspberry Pi 4** y la **tarjeta de cómputo original** del vehículo AWS DeepRacer. Contar con las dos plataformas operativas permite comparar su desempeño antes de decidir cuál se emplea en la demostración final, y proporciona una ruta de contingencia si alguna de las dos presenta limitaciones de cómputo al ejecutar la pila de navegación completa.

La instalación libera al vehículo de las restricciones del *firmware* de fábrica, que solo admite los modos de operación previstos por el fabricante y no expone el vehículo como un sistema ROS 2 accesible.

3.2. Verificación de acceso y locomoción

Se verificó el acceso por red a ambas tarjetas de cómputo y, a continuación, la locomoción del vehículo comandada desde su interfaz web. La prueba consistió en accionar el vehículo en avance, retroceso y giro, comprobando la respuesta del motor de tracción y del servomotor de dirección.

Tabla 1: Evidencia audiovisual de la prueba de marcha del vehículo.

Registro	Enlace
Prueba de marcha del AWS DeepRacer desde su interfaz web	https://youtu.be/ZGfAMnC4lYY

El registro documenta que el vehículo se encuentra **liberado y funcionando**: responde a los comandos de marcha enviados desde la interfaz web servida por el propio vehículo, sobre el entorno de cómputo instalado en esta semana [1].

3.3. Impacto sobre el objetivo específico 2

Esta actividad constituye el primer avance del proyecto sobre hardware real y eleva el objetivo específico OE2 del 25 % al 40 %. El desglose se presenta en la Tabla 2.

Tabla 2: Componentes de OE2 y su estado tras la Semana 17.

Componente	Estado	Observación
Locomoción	Cubierto	Verificada desde la interfaz web
Procesamiento	Cubierto	Entorno de cómputo instalado en ambas tarjetas
Comunicación (red)	Cubierto	Acceso por red a ambas tarjetas
Sensado	Pendiente	Requiere el LiDAR físico publicando sobre <code>/scan</code>
Control desde ROS 2	Pendiente	Requiere comandar <code>/cmd_vel</code> sin la interfaz web

Los dos componentes pendientes son precisamente el objeto del **diagnóstico de hardware** programado para la Semana 18, descrito en la Sección 9.

4. Consolidación del stack de navegación autónoma

4.1. Migración de Nav2 de ROS 2 Foxy a Humble

El stack de navegación desarrollado durante el primer periodo se encontraba sin publicar en el repositorio. Su consolidación requirió completar la migración entre distribuciones de ROS 2, cuyos cambios principales se resumen en la Tabla 3.

Tabla 3: Cambios requeridos por la migración de Nav2 de Foxy a Humble.

En Foxy	En Humble
<code>nav2_recoveries</code>	<code>nav2_behaviors</code>
<code>recoveries_server</code>	<code>behavior_server</code>
<code>smac_planner/SmacPlanner</code>	<code>nav2_smac_planner/SmacPlannerHybrid</code>
<code>robot_model_type:</code> <code>"differential"</code>	<code>robot_model_type:</code> <code>"nav2_amcl::DifferentialMotionModel"</code>
<code>default_bt_xml_filename</code>	<code>default_nav_to_pose_bt.xml</code> y <code>default_nav_through_poses_bt.xml</code>

4.2. Adecuación a cinemática Ackermann

Nav2 asume por defecto una plataforma diferencial, capaz de girar sobre su propio eje. El DeepRacer no puede hacerlo: su dirección es de tipo Ackermann y posee un radio de giro mínimo. Las adecuaciones realizadas se listan a continuación:

- **Árboles de comportamiento propios** sin la primitiva de giro sobre el eje, ya que el vehículo no puede ejecutarla y su invocación produce un fallo de recuperación.
- **Desactivación de la rotación hacia el rumbo** en el controlador, por la misma razón.
- **Corrección de la huella del vehículo** de $0,17 \times 0,17$ m a $0,28 \times 0,19$ m, ajustándola a las dimensiones reales del chasis.
- **Planificador con modelo de movimiento de radio mínimo**, configurado con un radio de giro de 0,35 m.

El valor configurado se contrastó con la geometría declarada en el modelo URDF del vehículo: ángulo máximo de dirección de $0,5236$ rad (30°) y distancia entre ejes de $0,164$ m, de donde se obtiene un **radio de giro mínimo real de $0,284$ m**. El valor de $0,35$ m empleado en la configuración es por tanto conservador, con un margen aproximado del 23% que absorbe las imprecisiones de seguimiento del controlador.

4.3. Publicación en el repositorio

El trabajo se publicó en el repositorio del proyecto en una rama de integración independiente, sobre la referencia remota de la rama principal. Se optó por una rama y no por la rama principal porque el historial local diverge del remoto y la rama principal es compartida con el segundo autor del proyecto; su incorporación queda sujeta a revisión conjunta.

5. Contrato de interfaces ROS 2

5.1. Motivación

Todo el trabajo restante del proyecto — nodo de coordinación, protocolo de relevo, interfaz de usuario e instrumentación de métricas — debe funcionar indistintamente sobre un modelo simulado en Gazebo y sobre un vehículo DeepRacer físico. Si esa equivalencia no se fija por escrito antes de escribir el código, cada módulo termina escribiéndose dos veces.

El contrato de interfaces congela, por tanto, la frontera entre módulos *antes* de implementarlos. Constituye el hito **H1** del cronograma actualizado y bloquea la totalidad del desarrollo posterior.

5.2. Contenido del contrato

Tabla 4: Interfaces por robot fijadas por el contrato.

Interfaz	Tipo	Función
/robotN/cmd_vel	Tópico	Comando de velocidad al vehículo
/robotN/odom	Tópico	Odometría publicada por el vehículo
/robotN/scan	Tópico	Lecturas del LiDAR
/robotN/joint_states	Tópico	Estado de las articulaciones
/robotN/navigate_to_pose	Acción	Único mecanismo de mando del coordinador
/robotN/estado	Tópico	Estado del agente para el coordinador
/robotN/initialpose	Tópico	Pose inicial para la localización

Las tres definiciones estructurales del contrato son:

1. **Todo cuelga de un espacio de nombres.** Únicamente `/tf`, `/tf_static`, `/clock` y `/rosout` permanecen en la raíz, por definición de ROS 2. Cualquier otro tópico en la raíz constituye una violación del contrato.
2. **Dos árboles de transformadas independientes**, con raíz en `robot1/map` y `robot2/map`, sin marco global compartido. La justificación es la decisión D2: ningún robot cruza entre niveles, de modo que la transición es un evento lógico del protocolo de relevo y no un desplazamiento físico. Un marco global compartido resolvería un problema que el sistema no tiene.
3. **El coordinador manda a un robot de una sola forma:** invocando su acción `navigate_to_pose`. No publica `cmd_vel`. Es precisamente esta restricción la que hace intercambiable un robot físico con uno simulado sin modificar el coordinador.

6. Línea base y verificación automática

6.1. Captura de la línea base

Antes de introducir cualquier cambio se capturó el comportamiento del sistema con un solo robot, para disponer de una referencia de regresión. Los resultados se presentan en la Tabla 5.

Tabla 5: Línea base del sistema con un robot, previa a la aplicación del contrato.

Elemento	Cantidad	Observación
Controladores activos	7 de 7	Confirma vigente la migración de S12
Tópicos publicados	27	Todos en la raíz, sin espacio de nombres
Marcos de referencia	13	Todos con nombre fijo, sin prefijo

Los trece marcos son cadenas fijas escritas en el modelo del vehículo. Con un segundo robot, los trece colisionan: este es el problema concreto que el contrato resuelve.

6.2. Instrumento de verificación

Se desarrolló una herramienta que comprueba de forma automática el cumplimiento del contrato sobre el sistema en ejecución. Opera en dos modos: verificación de salud con un solo robot, y verificación del contrato con dos robots en espacios de nombres separados.

La herramienta no se limita a listar los marcos de referencia. Recoge el conjunto de padres declarados para cada marco durante una ventana de observación, porque las herramientas estándar de visualización muestran un único padre por marco — el último recibido — y ocultan por completo el caso en que dos emisores se disputan el mismo marco.

6.3. Defecto detectado por la herramienta

En su primera ejecución la herramienta detectó, por sí sola, que el marco `camera_link` tenía **dos padres**: el modelo URDF lo declaraba hijo de `zed_camera_link`, mientras que un publicador de transformadas estáticas del archivo de lanzamiento lo declaraba hijo de `base_link`.

En el sistema de transformadas de ROS 2 un marco admite un solo padre, de modo que el resultado dependía del orden de llegada de los mensajes.

Al componer ambas cadenas se comprobó que las dos transformadas **no coinciden**: difieren en 15,39 mm en traslación, con rotación idéntica. No se trataba, por tanto, de una redundancia inocua.

El defecto se resolvió eliminando el publicador de transformadas estáticas del archivo de lanzamiento de simulación. El criterio fue que, en simulación, Gazebo posiciona el sensor a partir del modelo URDF, lo que convierte al URDF en la realidad física del sistema. El archivo de lanzamiento del vehículo físico se conserva sin cambios, puesto que allí no existe modelo URDF y esa transformada es la única fuente.

7. Corrección de la trazabilidad del entorno de compilación

7.1. Defecto detectado

El entorno de compilación del proyecto debe enlazar el repositorio versionado, no copiarlo, para garantizar que el código compilado sea el mismo que el código versionado. Se comprobó que esta condición **no se cumplía**: el directorio de fuentes del entorno de compilación seguía siendo una copia independiente, procedente de un clon del repositorio original del fabricante.

Una verificación previa había comprobado que el directorio de instalación enlazaba al de fuentes — lo cual era cierto — pero no que el directorio de fuentes fuese a su vez una copia. El defecto quedó así enmascarado.

7.2. Consecuencia sobre los resultados reportados

La consecuencia es directa y debe declararse: los ajustes de parámetros de SLAM realizados en la semana anterior **nunca llegaron a ejecutarse**. La simulación continuó operando con los valores originales. En particular, el rango máximo del LiDAR considerado por el algoritmo de mapeo siguió siendo de 12,0 m en lugar de los 9,5 m corregidos, valor este último que debe permanecer por debajo del alcance real del sensor.

Por tanto, **toda la evidencia cartográfica producida con anterioridad a esta corrección queda invalidada** como demostración del efecto de los parámetros corregidos, y el mapeo debe repetirse. Este hecho quedó registrado en el tablero de trazabilidad del proyecto.

7.3. Corrección aplicada

El directorio de fuentes se reemplazó por un enlace simbólico al repositorio versionado, previo respaldo del contenido anterior. La compilación posterior fue satisfactoria para los siete paquetes del proyecto. A partir de este punto, cualquier modificación del repositorio se refleja de forma inmediata en el código ejecutado.

8. Inicio de la aplicación del contrato

La aplicación del contrato sobre el código se abordó por pasos, bajo un criterio metodológico explícito: **cada paso se introduce con un valor por defecto que reproduce el comportamiento anterior, y se demuestra primero que no cambia nada**. De este modo, si algo falla con el valor por defecto, la causa está en la mecánica del cambio y no en el diseño del contrato.

8.1. Prefijo de marcos en el modelo del vehículo

De los trece marcos del vehículo, diez pueden ser prefijados por el nodo publicador del estado del robot mediante su parámetro correspondiente. Los tres restantes — `odom`, `base_link` y `laser` — no, porque son escritos directamente por complementos de Gazebo que ignoran ese parámetro. Estos tres se parametrizaron dentro del propio modelo del vehículo.

8.2. Espacio de nombres y pose inicial en el lanzamiento

Se incorporaron a los archivos de lanzamiento los argumentos de espacio de nombres y de pose inicial, este último necesario porque dos robots no pueden aparecer en el mismo punto del entorno. Con espacio de nombres vacío, el lanzamiento reproduce exactamente el comportamiento original.

8.3. Verificación

Tabla 6: Verificaciones ejecutadas sobre la aplicación del contrato.

Verificación	Resultado
Modelo del vehículo generado sin argumentos, contra la referencia previa	Idéntico línea a línea
Modelo generado con prefijo <code>robot1/</code>	Los tres marcos aparecen correctamente prefijados
Argumentos declarados por los archivos de lanzamiento	Los cinco argumentos nuevos con sus valores por defecto
Regresión en simulación con espacio de nombres vacío	6 de 6 tópicos, 7 de 7 controladores activos, 12 marcos con un solo padre

La última fila constituye la comprobación relevante: tras los cambios, el sistema con un solo robot se comporta igual que en la línea base, y el conflicto del marco `camera_link` ha desaparecido, lo que reduce el número de marcos de 13 a 12.

9. Estado del cronograma y trabajo previsto

9.1. Replanificación del cronograma

Durante esta semana se elaboró la replanificación completa del intervalo S17–S32, entregada como documento independiente [2]. Sus elementos principales son:

- Se mantienen las doce actividades del Capítulo 8 del anteproyecto, sin eliminar ni sustituir ninguna. Cambian únicamente sus ventanas temporales.
- El corrimiento neto de la parte técnica es de dos a cuatro semanas, originado en la migración no prevista de Nav2 y su adecuación a cinemática Ackermann.
- Dicho corrimiento se absorbe trasladando la redacción del documento final a la etapa posterior a la sustentación, conforme a la norma del espacio académico, lo que saca esa actividad del camino crítico.

- Se adoptan seis decisiones de alcance que delimitan qué se demuestra en simulación y qué sobre hardware, y que declaran expresamente las desviaciones que introducen.

9.2. Estado de actividades

Tabla 7: Estado de actividades — Semana 17 (Fase 5).

Actividad (anteproyecto)	Estado	Observación
Ajuste progresivo de los elementos desarrollados	En curso	Stack de navegación consolidado y publicado
Integración de los módulos del sistema	Iniciada	Contrato de interfaces definido y en aplicación
Registro de avances del proyecto	En curso	Presente entregable y cronograma actualizado

9.3. Trabajo previsto para la semana siguiente

Las actividades previstas para la Semana 18 son:

- **Diagnóstico acotado de hardware** (3 a 4 días), que responde cuatro preguntas y ninguna más: si el LiDAR real publica lecturas utilizables bajo la distribución instalada; si es posible comandar el vehículo desde ROS 2 sin pasar por la interfaz web; cuál es la latencia de comunicación entre los dos vehículos sobre la red del laboratorio; y qué implicaría emplear ROS 2 Jazzy en lugar de Humble.
- **Modelado del laboratorio GED** en formato SDF, duplicado en dos niveles con separación vertical, conforme a la decisión de alcance D4.
- **Construcción de la matriz de requisitos** que relaciona cada requisito funcional con su objetivo específico y su prueba de verificación.
- **Culminación de la aplicación del contrato:** espacio de nombres sobre el gestor de controladores y verificación de dos robots conviviendo en la misma simulación.

10. Conclusiones de la Semana

Durante la Semana 17 el proyecto obtuvo su primer resultado sobre hardware real: el vehículo AWS DeepRacer quedó liberado de su *firmware* original, con el entorno de cómputo instalado sobre dos plataformas y con su locomoción verificada desde la interfaz web, según consta en el registro audiovisual referenciado. Esto eleva el objetivo específico OE2 al 40 % y deja como únicos pendientes el sensado con el LiDAR físico y el control desde ROS 2.

En el frente de simulación se consolidó y publicó el stack de navegación autónoma, cerrando siete semanas de trabajo que permanecían sin versionar, y se definió el contrato de interfaces ROS 2 que constituye el hito H1 y habilita todo el desarrollo multi-robot posterior. Su aplicación sobre el código se inició de forma incremental y verificada, con una prueba de regresión satisfactoria.

Se detectó y corrigió, además, un defecto de trazabilidad en el entorno de compilación que mantenía desconectado el código versionado del código ejecutado. Su consecuencia — la invalidación de la evidencia cartográfica previa — se declara de forma explícita en este informe y obliga a repetir el mapeo. Se considera un hallazgo de valor: revela que ninguna corrección de parámetros anterior a esta semana había surtido efecto real sobre la simulación.

Finalmente, la replanificación del cronograma para el intervalo S17–S32 deja el proyecto con ventanas temporales realistas, criterios de cierre verificables por semana y una política explícita de gestión de la holgura que protege el aporte técnico central del proyecto.

Referencias

- [1] S. Hernández Ávila y J. A. Mejía León, “Liberación y prueba de marcha del AWS DeepRacer desde su interfaz web — Evidencia Semana 17,” Registro audiovisual, agosto de 2026. [Online]. Available: <https://youtu.be/ZGfAMnC4lYY>
- [2] S. Hernández Ávila y J. A. Mejía León, “Cronograma actualizado — Semanas 17 a 32,” Universidad Santo Tomás, Facultad de Ingeniería Electrónica, Bogotá, agosto de 2026.
- [3] S. Hernández Ávila y J. A. Mejía León, “Sistema colaborativo de robots móviles para asistencia de orientación en entornos interiores con múltiples pisos — Anteproyecto,” Universidad Santo Tomás, Facultad de Ingeniería Electrónica, Bogotá, 2026.

- [4] AWS DeepRacer Community, “deepracer-custom-car: Custom software stack for the AWS DeepRacer,” [Online]. Available: <https://github.com/aws-deepracer-community/deepracer-custom-car>
- [5] Open Robotics, “ROS 2 Humble Hawksbill Documentation,” [Online]. Available: <https://docs.ros.org/en/humble/>
- [6] Open Robotics, “Nav2 Documentation,” [Online]. Available: <https://docs.nav2.org/>
- [7] Open Robotics, “tf2: The transform library,” [Online]. Available: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html>
- [8] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” Journal of Open Source Software, vol. 6, no. 61, p. 2783, 2021. [Online]. Available: https://github.com/SteveMacenski/slam_toolbox